

TEST VE HATA AYIKLAMA

Yazılım geliştirmede test yazmak, kodun doğru çalıştığından emin olmak için kritik öneme sahiptir. Python'da test yazmak için `unittest` (standart kütüphane) ve `pytest` (üçüncü taraf, daha popüler) gibi araçlar kullanılır.

Test, kodunuzun beklediğiniz gibi çalıştığını doğrulayan küçük programlardır. Test yazmanın faydaları aşağıda sıralanmıştır:

- Hataları erken tespit eder
- Kod değişikliklerinde güven sağlar (refactoring)
- Kodun nasıl kullanılacağını gösterir (dokümantasyon)
- Daha iyi tasarım yapmaya zorlar
- Hata ayıklama süresini azaltır

Yazılım Doğrulamada Birim Testi

Yazılım mühendisliğinde bir fonksiyonun veya sınıfın beklenen işi yapıp yapmadığını otomatik olarak kontrol eden küçük test kodlarına **birim testi** (**unit test**) denir. Bir mühendis için test yazmak, tasarımın doğruluğunu ispatlamaktır. Kodun bir noktasında yapılan değişikliğin, sistemin başka bir yerini bozup bozmadığını (**regression**) anlamanın tek yolu budur.

Python, bu süreci yönetmek için standart kütüphanesinde `unittest` modülünü sunar.

unittest Modülü Temel Yapısı

Bir test dosyası oluştururken `unittest.TestCase` sınıfından kalıtım alan sınıflar yazılır. Test yöntemlerinin ismi mutlaka `test_` ifadesi ile başlamalıdır.

Örnek Senaryo olarak matematiksel bir fonksiyonun test edilmesini ele alalım.

```
# hesaplama.py
def toplama(a, b):
    return a + b
```

Şimdi bu fonksiyonu test eden sınıfımızı yazalım:

```
import unittest
from hesaplama import toplama

class TestMatematik(unittest.TestCase):
    def test_pozitif_toplama(self):
        # assertEquals: sonucun beklenen değere eşit olup olmadığını kontrol eder
        self.assertEqual(toplama(10, 5), 15)

    def test_negatif_toplama(self):
        self.assertEqual(toplama(-1, -1), -2)

if __name__ == '__main__':
    unittest.main()
```

İddia Yöntemleri

`unittest` içinde durumları kontrol etmek için çeşitli iddia yöntemleri (**assertion**) bulunur:

- `self.assertEqual(a, b)`: `a == b` olduğunu kontrol eder.
- `self.assertTrue(x)`: `x` değerinin `True` olduğunu kontrol eder.
- `self.assertRaises(İstisna)`: Kodun belirli bir istisnai duruma düşüp düşmediğini denetler.

Test Hazırlığı ve Temizliği

Her testten önce bir veri tabanı bağlantısı kurmak veya bir sensör nesnesi oluşturmak gerekebilir.

- `setUp()`: Her test metodundan önce otomatik çalışır.
- `tearDown()`: Her test metodundan sonra otomatik çalışır (temizlik işlemleri için).

```
class TestSensor(unittest.TestCase):
    def setUp(self):
        # Test öncesi sensör nesnesini hazırla
        self.sensor = Sensor(id=101)

    def test_okuma_araligi(self):
        deger = self.sensor.oku()
        self.assertTrue(0 <= deger <= 100)
```

pytest Modülü

Bu bölümde pytest'e odaklanacağız çünkü daha basit ve güçlüdür. Kurulum yapmak için `pip install pytest` komutu çalıştırılır.

pytest ile test dosyaları `test_` ile başlar veya `test` ile biter. Test fonksiyonları da `test` ile başlamalıdır. Hesaplamalar yapan bir `hesap.py` dosyasını modül olarak hazırlayalım;

```
# hesap.py
def topla(a: int, b: int) -> int:
    """İki sayıyı toplar."""
    return a + b

def carp(a: int, b: int) -> int:
    """İki sayıyı çarpar."""
    return a * b

def bol(a: float, b: float) -> float:
    """İki sayıyı böler."""
    if b == 0:
        raise ValueError("Sıfıra bölme hatası!")
    return a / b
```

Hesaplama dosyasının testini yapacak bir dosyayı aşağıdaki gibi hazırlayabiliriz;

```
# test_hesap.py
from hesaplama import topla, carp, bol
import pytest

def test_topla():
    """toplam fonksiyonunu test eder."""
    assert topla(2, 3) == 5
    assert topla(-1, 1) == 0
    assert topla(0, 0) == 0

def test_carp():
    """carp fonksiyonunu test eder."""
    assert carp(2, 3) == 6
    assert carp(-2, 3) == -6
    assert carp(0, 5) == 0

def test_bol():
    """bol fonksiyonunu test eder."""
    assert bol(6, 2) == 3.0
    assert bol(5, 2) == 2.5

def test_bol_sifira_bolme():
    """Sıfıra bölme hatasını test eder."""
```

```
with pytest.raises(ValueError):
    bol(10, 0)
```

Testleri çalıştırmak için `pytest test_hesaplama.py` komutu çalıştırılır. Çalıştırma sonucu aşağıdaki çıktı elde edilir;

```
D:\Python\PSamples> pytest test_hesap.py
===== test session starts =====
platform win32 -- Python 3.13.4, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\Python\PSamples
collected 4 items

test_hesap.py .... [100%]

===== 4 passed in 0.04s =====
D:\Python\PSamples>
```

assert Fonksiyonu

Bu fonksiyon yani `assert` (iddia etmek/doğrulamak) fonksiyonu, yazılımın hata toleransı ve savunmacı programlama (defensive programming) prensipleri çerçevesinde kullanılır.

Yazılım geliştirme sürecinde, belirli bir noktada bir koşulun mutlaka doğru (True) olması gerektiğini varsayabiliriz. `assert` fonksiyonu, bu varsayımların doğruluğunu çalışma anında (runtime) kontrol eden bir güvenlik mekanizmasıdır. Eğer belirtilen koşul yanlış (False) ise, program akışı anında kesilir ve bir `AssertionError` istisna nesnesi yaratılır. Söz dizimi aşağıda verilmiştir;

```
assert <koşul>, <hata_mesajı>
```

Bir mühendis için `assert`, kodun sadece düzgün çalışmasını değil, aynı zamanda beklenmedik durumlara karşı kendini imha etmesini (fail-fast) sağlar. Hatalı bir veriyle işleme devam edip sistemin geri dönülemez bir zarara uğraması yerine, hatanın kaynağında durdurulması tercih edilir. Aşağıdaki durumlarda kullanılabilir;

- Girdi Doğrulama: Bir fonksiyona gelen parametrelerin, mantıksal sınırlar içerisinde olup olmadığını denetler.
- Değişmezlerin (invariant) Kontrolü: Döngülerin başında veya sonunda bir değer asla değişmemesi gereken durumları denetler.
- Hata Ayıklama (debug): Kodun iç mantığını test etmek için kullanılır.

Bir personelin maaş artış oranını hesaplayan bir fonksiyonda, oran negatif olamaz. Bu, yazılımcının bir varsayımdır. Aşağıda buna ilişkin kod örneği verilmiştir;

```
def maas_artir(mevcut_maas, artis_orani):
    # Artis oranının negatif olamayacağını garanti altına alıyoruz
    assert artis_orani >= 0, "Artış oranı negatif olamaz!"

    yeni_maas = mevcut_maas * (1 + artis_orani)
    return yeni_maas

# Doğru kullanım
print(maas_artir(10000, 0.20))

# Hatalı kullanım (Program burada AssertionError verecektir)
print(maas_artir(10000, -0.10))
```

Yazılımcının yaptığı mantıksal hataları veya asla gerçekleşmemesi gereken durumları (programcı hataları) tespit etmek için `assert` fonksiyonu kullanılır. İstisna yönetimi için kullanılan `try-except` bloğu ise kullanıcı hataları (hatalı dosya yolu, yanlış şifre vb.) veya dış etkenler gibi beklenen hataları yönetmek için kullanılır.

Python yorumlayıcısı -O (**optimize**) bayrağı ile çalıştırıldığında tüm `assert` ifadeleri görmezden gelinir. Bu nedenle, **iş kritik** (**business-critical**) kontroller için `assert` yerine if blokları ve `raise` ifadeleri tercih edilmelidir. `assert` daha çok geliştirme ve test aşamasındaki iç denetim mekanizmasıdır.

`pytest` modülü Python'un dahili `assert` talimatını kullanır:

```
# Eşitlik kontrolü
assert 2 + 2 == 4

# Eşitsizlik kontrolü
assert 3 != 4

# Büyüklük kontrolü
assert 5 > 3
assert 3 < 5

# True/False kontrolü
assert True
assert not False

# Liste içeriği
assert [1, 2, 3] == [1, 2, 3]
assert 1 in [1, 2, 3]

# String içeriği
assert "python" in "python programlama"
assert "merhaba".startswith("mer")
```

Birden Fazla Girdiyle Test

Aynı testin farklı girdilerle çalıştırılması için **parametrelendirme** (**parametrize**) kullanılır:

```
# test_hesaplama.py
@pytest.mark.parametrize("a, b, beklenen", [
    (1, 2, 3),
    (5, 5, 10),
    (-1, 1, 0),
    (0, 0, 0),
    (100, 200, 300)
])
def test_topla_parametrize(a, b, beklenen):
    from hesaplama import topla
    assert topla(a, b) == beklenen
```

Bu testin çıktısı aşağıda verilmiştir;

```
$ pytest test_hesaplama.py::test_topla_parametrize -v
===== test session starts =====
test_hesaplama.py::test_topla_parametrize[1-2-3] PASSED      [ 20%]
test_hesaplama.py::test_topla_parametrize[5-5-10] PASSED     [ 40%]
test_hesaplama.py::test_topla_parametrize[-1-1-0] PASSED     [ 60%]
test_hesaplama.py::test_topla_parametrize[0-0-0] PASSED      [ 80%]
test_hesaplama.py::test_topla_parametrize[100-200-300] PASSED [====100%]

===== 5 passed in 0.05s =====
```

Testler için ortak bir veri ya da kaynak havuzu oluşturulabilir. Bu verilere **demirbaş** (**fixture**) adı verilir;

```
@pytest.fixture
def ornek_liste():
    """Test için örnek liste hazırlar."""
    return [1, 2, 3, 4, 5]

@pytest.fixture
```

```
def ornek_sozluk():
    """Test için örnek sözlük hazırlar."""
    return {"isim": "Ali", "yas": 25, "sehir": "Ankara"}

def test_liste_uzunlugu(ornek_liste):
    assert len(ornek_liste) == 5

def test_liste_toplam(ornek_liste):
    assert sum(ornek_liste) == 15

def test_sozluk_anahtarlari(ornek_sozluk):
    assert "isim" in ornek_sozluk
    assert "yas" in ornek_sozluk

def test_sozluk_degerleri(ornek_sozluk):
    assert ornek_sozluk["isim"] == "Ali"
    assert ornek_sozluk["yas"] == 25
```

Demirbaşların (**fixture**) ne zaman oluşturulup yok edileceğini belirlenebilir. Buna **demirbaş kapsamı** (**fixture scope**) adı verilir.

```
# Her test için yeni oluştur (varsayılan)
@pytest.fixture(scope="function")
def her_test_icin():
    print("\nHer test için oluşturuldu")
    return "veri"

# Tüm modül için bir kez oluştur
@pytest.fixture(scope="module")
def modul_icin_bir_kez():
    print("\nModül için bir kez oluşturuldu")
    veri = {"baglanti": "veritabani_baglantisi"}
    yield veri # yield kullanımı
    print("\nModül sonu - temizlik yapıldı")

# Tüm test oturumu için bir kez
@pytest.fixture(scope="session")
def oturum_icin_bir_kez():
    print("\nOturum başlangıcında oluşturuldu")
    return "global_veri"
```

Sahte Nesneler

Dış bağımlılıkları test etmek için **sahte** (**mock**) nesneler kullanılır;

```
# hava_durumu.py
import requests

def sehir_sicaklik(sehir: str) -> float:
    """Bir şehrin sıcaklığını API'den alır."""
    url = f"https://api.weather.com/{sehir}"
    yanit = requests.get(url)
    veri = yanit.json()
    return veri["sicaklik"]
```

```
# test_hava_durumu.py
from hava_durumu import sehir_sicaklik
from unittest.mock import Mock, patch

def test_sehir_sicaklik():
    """API çağrısını mock'luyoruz."""
```

```

with patch('hava_durumu.requests.get') as mock_get:
    # Mock yanıt oluştur
    mock_yanıt = Mock()
    mock_yanıt.json.return_value = {"sicaklik": 25.5}
    mock_get.return_value = mock_yanıt

    # Testi çalıştır
    sicaklik = sehir_sicaklik("Ankara")

    # Kontroller
    assert sicaklik == 25.5
    mock_get.assert_called_once_with("https://api.weather.com/Ankara")

```

pytest-mock Modülü

pytest-mock modülü daha kolay **sahte** (**mock**) nesne kullanımı sağlar. `pip install pytest-mock` komutu ile kurulum yapılır.

```

"""pytest-mock ile daha basit kullanım."""
# Mock oluştur
mock_get = mocker.patch('hava_durumu.requests.get')
mock_get.return_value.json.return_value = {"sicaklik": 30.0}

# Test
sicaklik = sehir_sicaklik("İstanbul")

# Kontrol
assert sicaklik == 30.0

```

Veri Tabanı Testi

Aşağıda **demirbaş** (**fixture**) yöntemi ile bir veri tabanı testi örneği verilmiştir;

```

import sqlite3

@pytest.fixture
def veritabani():
    """Test veritabanı oluşturur."""
    baglanti = sqlite3.connect(":memory:") # Bellekte veritabanı
    cursor = baglanti.cursor()

    # Tablo oluştur
    cursor.execute("""
        CREATE TABLE kullanicilar (
            id INTEGER PRIMARY KEY,
            isim TEXT NOT NULL,
            yas INTEGER
        )
    """)

    # Örnek veri ekle
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Ali', 25)")
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Ayşe', 30)")
    baglanti.commit()

    yield baglanti # Test için veritabanını ver

    baglanti.close() # Test sonrası temizle

def test_kullanici_sayisi(veritabani):
    cursor = veritabani.cursor()
    cursor.execute("SELECT COUNT(*) FROM kullanicilar")
    sayi = cursor.fetchone()[0]

```

```

    assert sayi == 2

def test_kullanici_ekle(veritabani):
    cursor = veritabani.cursor()
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Mehmet', 28)")
    veritabani.commit()

    cursor.execute("SELECT COUNT(*) FROM kullanicilar")
    sayi = cursor.fetchone()[0]
    assert sayi == 3

```

Test Kapsama Oranı

Kodunuzun ne kadarının test edildiğini görmek için **test kapsama oranı** (coverage) kullanılır. `pip install pytest-cov` konutu ile kurulum yapılır.

```

$ pytest --cov=hesaplama test_hesap.py
===== test session starts =====
collected 4 items

test_hesap.py .... [100%]

----- coverage: platform win32, python 3.10.0 -----
Name           Stmts   Miss  Cover
-----
hesap.py         8      0   100%
-----
TOTAL            8      0   100%

===== 4 passed in 0.10s =====

```

HTML raporu oluşturmak için `pytest --cov=hesaplama --cov-report=html test_hesap.py` komutu kullanılabilir.

Test Kapsama Oranı (Coverage) Hedefleri

% 80 ile 100 arası mükemmel,
 %60 ile 80 arası iyi,
 %40 ile 60 arası kabul edilebilir,
 %0 ile 40 arası yetersiz sayılır.

Testleri Gruplandırma

Testleri işaretleyerek gruplandırabiliriz.

```

@pytest.mark.hizli
def test_hizli_islem():
    assert 1 + 1 == 2

@pytest.mark.yavas
def test_yavas_islem():
    import time
    time.sleep(2)
    assert True

@pytest.mark.veritabani
def test_veritabani_baglantisi():
    # Veritabanı testi
    pass

```

```
@pytest.mark.network
def test_api_cagrisi():
    # API testi
    pass
```

Bu testleri çalıştırmak için aşağıdaki komutlar çalıştırılabilir;

```
$ pytest -m hizli # Sadece hızlı testler
$ pytest -m "not yavas" # Yavaş testler hariç
$ pytest -m "veritabani or network" # Veritabanı veya network testleri
```

Paylaşımlı Demirbaşlar

Aşağıdaki dosyadaki **demirbaşlar** (**fixture**) tüm test dosyalarında kullanılabilir.

```
# conftest.py
import pytest

@pytest.fixture
def uygulama_ayarlari():
    """Tüm testlerde kullanılabilir."""
    return {
        "veritabani": "test.db",
        "debug": True,
        "port": 5000
    }

@pytest.fixture(autouse=True)
def test_baslangic_bitis():
    """Her testten önce ve sonra otomatik çalışır."""
    print("\n--- Test başlıyor ---")
    yield
    print("\n--- Test bitti ---")
```

Zaman Uyumsuz Fonksiyon Testleri

Zaman uyumsuz (**asynchron**) fonksiyonları test etmek için `pytest-asyncio` kullanılır. `pip install pytest-asyncio` komutu ile kurulum yapılır.

```
# async_islemler.py
import asyncio

async def asenkron_topla(a: int, b: int) -> int:
    await asyncio.sleep(0.1) # Simüle edilmiş asenkron işlem
    return a + b

async def api_cagir(url: str) -> dict:
    await asyncio.sleep(0.5)
    return {"durum": "basarili", "veri": "test"}
```

Bu işlemlerin bulunduğu dosyayı test etmek için aşağıdaki kod yazılabilir;

```
# test_async_islemler.py
import pytest
from async_islemler import asenkron_topla, api_cagir

@pytest.mark.asyncio
async def test_asenkron_topla():
    sonuc = await asenkron_topla(2, 3)
    assert sonuc == 5

@pytest.mark.asyncio
async def test_api_cagir():
    sonuc = await api_cagir("https://api.example.com")
    assert sonuc["durum"] == "basarili"
```


Test Odaklı Yazılım Geliştirme

Test odaklı yazılım geliştirme (**Test Driven Development-TDD**), önce test yazıp sonra kodu yazma metodolojisidir. Bu metodolojide ilk önce başarısız bir test yazılır (**red**), sonrasında testi geçecek minimum kod yazılır (**green**), en sonunda kod iyileştirilir (**refactor**).

```
# 1. Önce test yaz (başarısız olacak)
def test_kullanici_olustur():
    kullanıcı = Kullanici("Ali", "ali@example.com")
    assert kullanıcı.isim == "Ali"
    assert kullanıcı.email == "ali@example.com"

# 2. Kodu yaz (testi geçirecek)
class Kullanici:
    def __init__(self, isim: str, email: str):
        self.isim = isim
        self.email = email

# 3. Kodu iyileştir
class Kullanici:
    def __init__(self, isim: str, email: str):
        if not isim:
            raise ValueError("İsim boş olamaz")
        if "@" not in email:
            raise ValueError("Geçersiz email")
        self.isim = isim
        self.email = email
```

En İyi Uygulamalar

- Her fonksiyon için en az bir test yazılmalı
- Test isimleri açıklayıcı olmalı (`test_topla_iki_pozitif_sayi`)
- Testler birbirinden bağımsız olmalı ve hızlı çalışmalı
- Sahte nesne (**mock**) kullanarak dış bağımlılıkları izole edilmeli
- Test kapsama oranı (**coverage**) %100'e yakın olmalı (kritik kod için)
- Demirbaşlar (**fixture**) tekrar kullanılabilir yapılmalı
- Testleri her sürüm sırasında düzenli çalıştırılmalı (CI/CD)
- Test kodu da temiz tutulmalı

Test İçeren Örnek Proje

Test içeren örnek bir proje aşağıdaki gibi olmalıdır;

```
proje/
├── src/
│   ├── __init__.py
│   ├── hesaplama.py
│   └── kullanıcı.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_hesaplama.py
│   └── test_kullanici.py
├── pytest.ini
└── requirements.txt
```

Bu projede yer alan `pytest.ini` dosyası aşağıdaki gibi olmalıdır;

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
```

```
markers =  
    hizli: Hızlı testler  
    yavas: Yavaş testler  
    veritabani: Veritabanı testleri  
    network: Network gerektiren testler
```